

GUÍA COMPLETA DE JAVAFX (DE BÁSICO A AVANZADO)

1. FUNDAMENTOS: Stage, Scene y Root

Antes de mostrar cualquier cosa en pantalla, JavaFX funciona como un teatro:

Conceptos clave

- **Stage (Escenario)** → Es la ventana (cerrar, minimizar, etc.)
 - **Scene (Escena)** → El contenido dentro de la ventana
 - **Root Node (Nodo raíz)** → Contenedor principal de todos los elementos
-

Estructura básica

```
public class App extends Application {  
  
    @Override  
    public void start(Stage primaryStage) {  
  
        StackPane root = new StackPane(); // nodo raíz  
  
        Scene scene = new Scene(root, 600, 400);  
  
        primaryStage.setTitle("Sesión de: " + nombreUsuario);  
        primaryStage.setScene(scene);  
        primaryStage.setResizable(false); // evitar redimensionar  
        primaryStage.show(); // mostrar ventana  
    }  
}
```

Clave:

- `setResizable(false)` → bloquea el tamaño de la ventana
-



2. NODOS Y CONTROLES (LOS ACTORES)

Todo lo visible en JavaFX es un **Node**.



Controles básicos



Label

- Muestra texto

```
Label etiqueta = new Label("Hola");  
etiqueta.setText("Nuevo texto");
```



Button

- Permite interacción

```
Button boton = new Button("Click");
```



Eventos (lo importante)

```
boton.setOnAction(e -> {  
    System.out.println("¡El botón funciona!");  
});
```



Agregar al root

```
root.getChildren().addAll(etiqueta, boton);
```



3. LAYOUTS (ORDENAR LA INTERFAZ)



Problema

`StackPane` apila todo (queda superpuesto)



Solución: Layouts

HBox (horizontal)

```
HBox root = new HBox();
```

VBox (vertical)

```
VBox root = new VBox();
```

Propiedades clave

```
root.setSpacing(10); // espacio entre elementos  
root.setAlignment(Pos.CENTER); // centrado
```

Ejemplo completo

```
VBox root = new VBox();  
  
Label textoPr = new Label("Prueba");  
Button botonAccept = new Button("Aceptar");  
  
root.getChildren().addAll(textoPr, botonAccept);  
root.setSpacing(20);  
root.setAlignment(Pos.CENTER);
```

4. ENTRADA DE DATOS (TextField)

Capturar texto

```
TextField campo = new TextField();  
String texto = campo.getText();
```

Validación

```
if (campo.getText().trim().isEmpty()) {  
    System.out.println("Vacío");  
}
```

Alertas

```
Alert alerta = new Alert(Alert.AlertType.WARNING);  
alerta.setTitle("Error");  
alerta.setContentText("Campo vacío");
```

```
alerta.showAndWait();
```

5. LÓGICA COMPLETA (EJEMPLO REAL)

Mini app de saludo

```
botonSaludo.setOnAction(e -> {  
  
    if (campoNombre.getText() == null || campoNombre.getText().trim().isEmpty()) {  
  
        Alert alerta = new Alert(Alert.AlertType.WARNING);  
        alerta.setTitle("Campo obligatorio");  
        alerta.setContentText("Ingresa un nombre");  
        alerta.showAndWait();  
  
    } else {  
        String saludo = campoNombre.getText();  
        textSaludo.setText("Saludando a " + saludo);  
    }  
});
```

Lo que aprendés acá:

- Validación (`trim().isEmpty()`)
 - Manejo de estado (actualizar Label)
 - Interacción real
-

6. GRIDPANE (INTERFACES MÁS COMPLEJAS)

Cuando VBox no alcanza → usamos grilla

Coordenadas

```
grid.add(nodo, columna, fila);
```

Ejemplo:

```
grid.add(textCelsius, 0, 0);  
grid.add(entradaCel, 1, 0);
```

Configuración

```
grid.setHgap(10);  
grid.setVgap(10);  
grid.setPadding(new Insets(20));  
grid.setAlignment(Pos.CENTER);
```

Alineación individual

```
GridPane.setHalignment(botonCalc, HPos.RIGHT);
```

7. PARSING (STRING → NÚMERO)

Todo lo que escribe el usuario es String

```
double num = Double.parseDouble(texto);
```

Corrección de comas:

```
texto.replace(",", ".");
```

8. EJEMPLO COMPLETO (CONVERSOR)

```
try {  
    double temperatura = Double.parseDouble(entradaCel.getText().replace(",", "."));  
    double resultadoF = (temperatura * 1.8) + 32;  
  
    resultado.setText(String.format("%.2f °F", resultadoF));  
  
} catch (NumberFormatException e) {  
  
    Alert alerta = new Alert(Alert.AlertType.ERROR);  
    alerta.setContentText("Solo números");  
    alerta.showAndWait();  
}
```

```
entradaCel.clear();
entradaCel.requestFocus();
resultado.setText("Entrada inválida");
}
```

9. CSS EN JAVAFX

Formas de usarlo

Inline

```
boton.setStyle("-fx-background-color: red;");
```

Archivo externo (mejor)

```
scene.getStylesheets().add("estilos.css");
```

Selectores

- `#id` → elemento único
 - `.clase` → grupo
-

10. IMÁGENES

Image

```
Image img = new Image("file:foto.png");
```

ImageView

```
ImageView vista = new ImageView(img);
```

```
vista.setFitWidth(200);
vista.setPreserveRatio(true);
```

Métodos clave

- `setImage()`
 - `setRotate()`
 - `setVisible()`
-



11. LISTVIEW (LISTAS DINÁMICAS)



Operaciones

```
lista.getItems().add("Tarea");  
lista.getItems().remove(0);  
lista.getItems().clear();
```



Selección

```
String tarea = lista.getSelectionModel().getSelectedItem();  
int index = lista.getSelectionModel().getSelectedIndex();
```



Eventos

```
lista.setOnMouseClicked(e -> {  
    if (e.getClickCount() == 2) {  
        System.out.println("Doble click");  
    }  
});
```



Editar / eliminar

```
lista.getItems().remove(index);  
lista.getItems().set(index, "Nueva tarea");
```



12. BUTTON (PROFUNDIZACIÓN)



Eventos

```
button.setOnAction(...)  
button.fire()
```



Estado

```
button.setDisable(true);
```



Tamaño

```
button.setPrefWidth(100);
```



Tooltip

```
button.setTooltip(new Tooltip("Click"));
```



13. FXML (SEPARACIÓN PROFESIONAL)



División

- **FXML** → interfaz
 - **Controller** → lógica
 - **CSS** → estilos
-



14. SCENE BUILDER

Herramienta visual (drag & drop)



Triángulo

- **FXML** → estructura
 - **Controller** → comportamiento
 - **Application** → carga
-



15. CONTROLLER

```
public class PrincipalController {
```



```
@FXML
private Button btnHola;
```

```
@FXML
void saludar() {
    btnHola.setText("¡Presionado!");
}
}
```

Vinculación en FXML

```
<Button fx:id="btnHola" onAction="#saludar"/>
```

16. CARGAR FXML (FXMLLoader)

```
FXMLLoader loader = new FXMLLoader(
    getClass().getResource("/vista/VistaPrincipal.fxml")
);
```

```
Parent root = loader.load();
```

```
Scene scene = new Scene(root);
primaryStage.setScene(scene);
primaryStage.show();
```

Qué pasa acá:

- **FXMLLoader** → lee el FXML
 - crea nodos
 - conecta Controller
-

Parent

Es el nodo raíz del árbol:

```
VBox
├── Label
└── Button
```

CONCLUSIÓN (TU PROGRESIÓN)

Con todo esto pasaste por:

1.  Ventana básica
2.  Controles
3.  Layouts
4.  Entrada de datos
5.  Lógica real
6.  Interfaces complejas
7.  Parsing
8.  Manejo de errores
9.  Estilos
10.  Multimedia
11.  Listas dinámicas
12.  Control avanzado
13.  Arquitectura profesional (FXML)